Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Санкт-Петербургский национальный исследовательский университет информационных технологий, механики и оптики»

Факультет «Программной Инженерии и Компьютерной Техники»

Дисциплина «Системное программное обеспечение»

Отчёт к практическому заданию № 1

Вариант 2

Входной язык 2

Выполнил:

Студент группы P4114 Тягельский А. А.

Проверил:

Доцент факультета ПИиКТ Кореньков Юрий Дмитриевич

Санкт-Петербург, 2025

Цель работы:

Использовать средство синтаксического анализа по выбору, реализовать модуль для разбора текста в соответствии с языком по варианту. Реализовать построение по исходному файлу с текстом синтаксического дерева с узлами, соответствующими элементам синтаксической модели языка. Вывести полученное дерево в файл в формате, поддерживающем просмотр графического представления.

Задачи:

1. Выбрать средство синтаксического анализа, совместимое с языком C, которое позволит по заданной грамматике построить дерево разбора.
2. Описать в файле Grammar.g спецификацию для средства синтаксического анализа (грамматику языка), включающую конструкции, заданные вариантом.
3. По заданной грамматике с помощью средства синтаксического анализа сгенерировать GrammarLexer и GrammarParser.
4. Реализовать модуль, использующий средство синтаксического анализа для разбора языка. Программа принимает на вход путь к файлу с кодом, а также путь к файлу/директории, в которую необходимо положить результат (дерево разбора), в консоль выводятся сообщения об ошибке.
5. Разработать тест кейсы для демонстрации работоспособности созданного модуля, проверить правильность работы созданного модуля.

Ход работы

1 Выбор средства синтаксического анализа

В данной работе для синтаксического анализа языка выбран фреймворк ANTLR3, так как он предоставляет полноценную поддержку языка C и позволяет автоматически генерировать лексер и парсер на основе формального описания грамматики. ANTLR3 поддерживает построение дерева разбора (AST), что упрощает дальнейшие этапы анализа и обработки программы, такие как семантический анализ или генерация кода. В качестве альтернативы рассмотрен Bison, однако он требует более детальной проработки грамматики, помимо этого учтены мнения одногруппников, которые пробовали оба фреймворка и выбирали именно ANTLR3. К сожалению, ANTLR4 не поддерживает язык C, поэтому приходится использовать более старую версию фреймворка.

2 Описание спецификации для средства синтаксического анализа

Для описания грамматики используется файл Grammar.g, в котором при помощи разметки определяются токены, лексика и правила парсинга. В результате получилась следующая грамматика:

```
grammar Grammar;
options {
   language = C;
   output = AST;
}
tokens {
   SOURCE;
   METHOD_DECL;
   PARAMETERS;
   PARAMETER;
   TYPE;
   VOID_VALUE;
   RETURN_TYPE;
   BODY;
   VAR_DECLARATIONS;
   VAR_DECLARATION;
   VARIABLES;
   BLOCK;
   EXPRESSION;
   CALL;
   ASSIGN;
```

```
    VALUE;
    ARGUMENTS;
    IN_BRACES;
    UNARY_OPERATION;
    ARRAY_ELEMENT;
    ARRAY_ID;
    ARRAY_ELEMENT_INDEX;
    ADD;
    SUBTRACT;
    MULTIPLY;
    DIVISION;
    RESIDUE;
    EQUALS;
    NOT_EQUALS;
    LESS_THAN;
    LESS_THAN_OR_EQUALS;
    MORE_THAN;
    MORE_THAN_OR_EQUALS;
    AND;
    OR;
    ID;
    IF;
    CONDITION;
    THEN;
    ELSE;
    WHILE;
    DO;
    REPEAT;
    REPEATABLE_PART;
    BREAK;
    UNTIL;
}

/* ============================
   LEXER
   ============================ */

fragment LETTER: 'a'..'z'|'A'..'Z'|'_';
fragment DIGIT: '0'..'9';

WS: (' '|'\t'|'\r'|'\n')+ { $channel=HIDDEN; };

K_TRUE: 'true';
K_FALSE: 'false';
K_METHOD: 'method';
K_VAR: 'var';
K_IF: 'if';
K_THEN: 'then';
K_ELSE: 'else';
K_BEGIN: 'begin';
```

```
K_END: 'end';
K_WHILE: 'while';
K_DO: 'do';
K_REPEAT: 'repeat';
K_UNTIL: 'until';
K_BREAK: 'break';
K_OF: 'of';
K_ARRAY: 'array';
K_BOOL: 'bool';
K_BYTE: 'byte';
K_INT: 'int';
K_UINT: 'uint';
K_LONG: 'long';
K_ULONG: 'ulong';
K_CHAR: 'char';
K_STRING: 'string';

IDENTIFIER: LETTER (LETTER|DIGIT)*;

STRING: '"' ( ~('"'|'\\') | '\\' . )* '"';
CHAR: '\'' ( ~('\'|'\\') | '\\' . ) '\'';
HEX: '0' ('x'|'X') ('0'..'9'|'a'..'f'|'A'..'F')+;
BITS: '0' ('b'|'B') ('0'|'1')+;
DEC: DIGIT+;

/* ============================
  PARSER
  ============================ */

source
    : sourceItem* EOF
      -> ^(SOURCE sourceItem*)
    ;

sourceItem: funcDef;

typeRef
    : builtinType
    | IDENTIFIER
    | K_ARRAY '[' commaList ']' K_OF typeRef
    ;

commaList: (',' )*;

builtinType
    : K_BOOL | K_BYTE | K_INT | K_UINT | K_LONG | K_ULONG | K_CHAR |
K_STRING
    ;

funcDef:
```

```
  K_METHOD funcSignature (methodBody | ';')
    -> ^(METHOD_DECL funcSignature methodBody?)
  ;

funcSignature: methodName '(' methodParameters? ')' methodReturnType -> methodName
methodParameters? methodReturnType;

methodName:
  IDENTIFIER -> ^(ID IDENTIFIER)
  ;

methodParameter: IDENTIFIER (':' typeRef)? -> ^(PARAMETER ^(ID IDENTIFIER)
^(TYPE typeRef)?);
methodParameters: methodParameter (',' methodParameter)* -> ^(PARAMETERS
methodParameter+);

methodReturnType:
  ':' typeRef -> ^(RETURN_TYPE typeRef)
  | -> ^(RETURN_TYPE VOID_VALUE)
  ;

methodBody:
  (K_VAR varDeclaration*)? statementBlock -> ^(BODY ^(VAR_DECLARATIONS
varDeclaration*)? statementBlock)
  ;

varDeclaration:
  variableNameList (':' typeRef)? ';' -> ^(VAR_DECLARATION ^(TYPE typeRef)?
variableNameList)
  ;

variableNameList:
  IDENTIFIER (',' IDENTIFIER)* -> ^(VARIABLES (^(ID IDENTIFIER))+)
  ;

/* ============================
  STATEMENTS
  ============================ */

statement
  : ifStatement
  | whileStatement
  | repeatStatement
  | breakStatement
  | statementBlock
  | expressionStatement
  ;

ifStatement
  : K_IF expression K_THEN statement (K_ELSE statement)?
```

```
    -> ^(IF ^(CONDITION expression) ^(THEN statement) ^(ELSE statement)?)
  ;

whileStatement
  : K_WHILE expression K_DO statement
    -> ^(WHILE ^(CONDITION expression) ^(DO statement))
  ;

repeatStatement
  : K_REPEAT statement (K_WHILE | K_UNTIL) expression ';'
    -> ^(REPEAT ^(REPEATABLE_PART statement) ^(UNTIL expression))
  ;

breakStatement
  : K_BREAK ';'
    -> ^(BREAK)
  ;

statementBlock:
  K_BEGIN statement* K_END ';' -> ^(BLOCK statement*)
  ;

expressionStatement:
  expression ';' -> expression
  ;

/* ============================
  EXPRESSIONS
  ============================ */

expression:
  assignExpression
  | calculationExpression -> ^(EXPRESSION calculationExpression);

assignExpression:
  IDENTIFIER ':=' calculationExpression -> ^(ASSIGN ^(ID IDENTIFIER) ^(VALUE
calculationExpression));

calculationExpression:
  orExpression;

orExpression:
  (andExpression '||' orExpression) => a=andExpression ('||' b=orExpression)+ -> ^(OR $a
$b+)
  | andExpression
  ;

andExpression:
  (eqExpression '&&' andExpression) => a=eqExpression ('&&' b=andExpression)+ ->
^(AND $a $b+)
```

```
    | eqExpression
    ;

eqExpression:
    (relExpression equalOperator eqExpression) => relExpression equalOperator eqExpression
-> ^(equalOperator relExpression eqExpression)
    | relExpression
    ;

equalOperator:
     '==' -> ^(EQUALS)
     | '!=' -> ^(NOT_EQUALS);

relExpression:
    (addExpression compareOperator relExpression) => addExpression compareOperator
relExpression -> ^(compareOperator addExpression relExpression)
    | addExpression
    ;

compareOperator:
     '<' -> ^(LESS_THAN)
     | '<=' -> ^(LESS_THAN_OR_EQUALS)
     | '>' -> ^(MORE_THAN)
     | '>=' -> ^(MORE_THAN_OR_EQUALS);

addExpression:
    (mulExpression plusSubtractOperator addExpression) => mulExpression
plusSubtractOperator addExpression -> ^(plusSubtractOperator mulExpression
addExpression)
    | mulExpression
    ;

plusSubtractOperator:
     '+' -> ^(ADD)
     | '-' -> ^(SUBTRACT);

mulExpression:
    (unaryExpression mulOperator mulExpression) => unaryExpression mulOperator
mulExpression -> ^(mulOperator unaryExpression mulExpression)
    | unaryExpression
    ;

mulOperator:
     '*' -> ^(MULTIPLY)
     | '/' -> ^(DIVISION)
     | '%' -> ^(RESIDUE);

unaryExpression:
    unaryOp unaryExpression -> ^(UNARY_OPERATION unaryOp unaryExpression)
    | primaryExpr;
```

```
unaryOp:
    '+'
    | '-'
    | '!';

primaryExpr:
    callExpression
    | bracesExpression
    | indexerExpression
    | placeExpression
    | literal;

callExpression:
    IDENTIFIER '(' argumentList? ')' -> ^(CALL ^(ID IDENTIFIER) argumentList?);

argumentList:
    calculationExpression (',' calculationExpression)* -> ^(ARGUMENTS
calculationExpression+);

bracesExpression:
    '(' calculationExpression ')' -> ^(IN_BRACES calculationExpression);

indexerExpression:
    IDENTIFIER '[' calculationExpression ']' -> ^(ARRAY_ELEMENT ^(ARRAY_ID
IDENTIFIER) ^(ARRAY_ELEMENT_INDEX calculationExpression));

placeExpression:
    IDENTIFIER -> ^(ID IDENTIFIER);

literal:
    K_TRUE   -> K_TRUE
    | K_FALSE  -> K_FALSE
    | STRING   -> STRING
    | CHAR     -> CHAR
    | HEX      -> HEX
    | BITS     -> BITS
    | DEC      -> DEC
    ;
```

3 Генерация кода для парсинга языка по грамматике

Для генерации кода по грамматике использовалась программа на языке java «antlr-3.4-complete.jar», которая по грамматике из файла Grammar.g, подаваемого в виде входного параметра, создаёт файлы Grammar.tokens, GrammarLexer.c, GrammarLexer.h, GrammarParser.c и GrammarParser.h. Используя указанные файлы можно парсить язык и получать дерево разбора по заданным в грамматике правилам.

4 Реализация модуля для разбора языка

Для предоставления API работы с модулем разбора создан файл parser_module.h с описанием структуры данных, являющейся результатом работы модуля. Структура ParseResult позволяет получить доступ к дереву разбора, набору ошибок и объектам ANTLR, которые необходимо освободить после прекращения работы с деревом разбора. Модуль предоставляет следующие методы для открытого использования из main: метод parseFile, который по пути к файлу анализирует код и возвращает ParseResult структуру; freeParseResult для удаления объектов после прекращения использования дерева разбора; printTree для вывода дерева разбора на консоль; treeToDot для вывода дерева разбора в .dot файл, который можно визуализировать при помощи Graphviz. Ниже использованный код:

```
typedef struct {
    pANTLR3_BASE_TREE tree;          // AST root
    char** errors;                   // array of error messages
    int errorCount;

    // ANTLR objects needed for proper cleanup
    pGrammarParser parser;
    pANTLR3_COMMON_TOKEN_STREAM tokens;
    pGrammarLexer lexer;
    pANTLR3_INPUT_STREAM input;
} ParseResult;

ParseResult parseFile(const char* filename);
void freeParseResult(ParseResult* result);
void printTree(pANTLR3_BASE_TREE tree, int indent);
void treeToDot(pANTLR3_BASE_TREE tree, FILE* dotFile);
```

Код реализации всех данных методов предоставлен в файле parser_module.c, который реализует логику создания дерева разбора по предоставляемому коду, а также сбор ошибок, вывод дерева разбора в консоль и в файл .dot. Код модуля представлен ниже:

```
#include "parser_module.h"

#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
```

```
// Custom ANTLR error collector
static void collectError(pANTLR3_BASE_RECOGNIZER recognizer,
                         pANTLR3_UINT8* tokenNames)
{
    pANTLR3_EXCEPTION ex = recognizer->state->exception;
    recognizer->state->errorCount++;

    const char* msg = (const char*)ex->message;

    char*** listPtr = (char***)&recognizer->state->userp;
    char** list = *listPtr;

    int index = recognizer->state->errorCount - 1;

    list[index] = malloc(strlen(msg) + 1);
    strcpy(list[index], msg);
}

ParseResult parseFile(const char* filename)
{
    ParseResult result;
    result.tree = NULL;
    result.errorCount = 0;
    result.errors = calloc(128, sizeof(char*));

    result.parser = NULL;
    result.tokens = NULL;
    result.lexer = NULL;
    result.input = NULL;

    // Создаем ANTLR file stream
    pANTLR3_INPUT_STREAM input = antlr3FileStreamNew(
        (pANTLR3_UINT8)filename,
        ANTLR3_ENC_8BIT
    );
    if (!input) {
        result.errorCount = 1;
        result.errors[0] = strdup("Failed to open input file.");
        return result;
    }

    // Лексер
    pGrammarLexer lexer = GrammarLexerNew(input);
```

```
    // Токены
    pANTLR3_COMMON_TOKEN_STREAM tokens =
        antlr3CommonTokenStreamSourceNew(ANTLR3_SIZE_HINT,
TOKENSOURCE(lexer));

    // Парсер
    pGrammarParser parser = GrammarParserNew(tokens);

    // Error listener
    parser->pParser->rec->displayRecognitionError = collectError;
    parser->pParser->rec->state->userp = result.errors;

    // Парсим
    GrammarParser_source_return ret = parser->source(parser);
    result.tree = ret.tree;
    result.errorCount = parser->pParser->rec->state->errorCount;

    // Сохраняем объекты для очистки в main
    result.parser = parser;
    result.tokens = tokens;
    result.lexer = lexer;
    result.input = input;

    return result;
}

void freeParseResult(ParseResult* result)
{
    for (int i = 0; i < result->errorCount; i++)
        free(result->errors[i]);
    free(result->errors);

    if (result->parser)
        result->parser->free(result->parser);
    if (result->tokens)
        result->tokens->free(result->tokens);
    if (result->lexer)
        result->lexer->free(result->lexer);
    if (result->input)
        result->input->close(result->input);
}
```

```
// Рекурсивный вывод дерева
void printTree(pANTLR3_BASE_TREE tree, int indent) {
    if (!tree) return;

    for (int i = 0; i < indent; i++)
        printf("  ");

    pANTLR3_STRING text = tree->getText(tree);
    printf("%s\n", text->chars);

    ANTLR3_UINT32 childCount = tree->getChildCount(tree);
    for (ANTLR3_UINT32 i = 0; i < childCount; i++) {
        pANTLR3_BASE_TREE child = (pANTLR3_BASE_TREE)tree-
>getChild(tree, i);
        printTree(child, indent + 1);
    }
}


void treeNodeToDot(pANTLR3_BASE_TREE tree, FILE* dotFile, int*
nodeId)
{
    int currentId = (*nodeId)++;
    pANTLR3_STRING text = tree->getText(tree);
    fprintf(dotFile, "  node%d [label=\"%s\"];\n", currentId, text->chars);

    ANTLR3_UINT32 childCount = tree->getChildCount(tree);
    for (ANTLR3_UINT32 i = 0; i < childCount; i++) {
        int childId = *nodeId;
        pANTLR3_BASE_TREE child = tree->getChild(tree, i);

        treeNodeToDot(child, dotFile, nodeId);
        fprintf(dotFile, "  node%d -> node%d;\n", currentId, childId);
    }
}


void treeToDot(pANTLR3_BASE_TREE tree, FILE* dotFile)
{
    fprintf(dotFile, "digraph AST {\n  node [shape=box];\n");
    int nodeId = 0;
    treeNodeToDot(tree, dotFile, &nodeId);
```

```
    fprintf(dotFile, "}\n");
}
```

Файл для запуска программы использует разработанный модуль для получения дерева разбора и набора ошибок, после чего выводит дерево разбора в консоль и в файл .dot. Если ошибки присутствуют, но дерево разбора построено, то ошибки просто выводятся на консоль, если дерево разбора не построено из-за ошибок, то выводится соответствующий текст с ошибкой.

5 Тестирование модуля на работоспособность

Для тестирования модуля на работоспособность разработано более 30 тест кейсов, которые проверяют парсер.

1.

```
method hello();
```

2.

```
method hello(x, y : int, z : myType);
```

3.

```
method hello(x, y : int) : bool;
```

4.

```
method a();
method b(x:int) : int;
method c(x, y, z : long);
```

5.

```
method m() : int
begin
end;
```

6.

```
method m()
var a, b : int;
c : bool;
begin
end;
```

7.

```
method m()
begin
1 + 2;
end;
```

8.

```
method m()
begin
x := 10;
end;
```

9.

```
method m()
begin
x := (1 + 2) * 3;
end;
```

10.

```
method m()
begin
x := -1;
y := !true;
```

```
z := +5;
end;
```

11.

```
method m()
begin
x := 1 < 2;
y := 2 >= 3;
z := 4 == 5;
t := 6 != 7;
end;
```

12.

```
method m()
begin
x := a && b && c;
y := a || b || c;
end;
```

13.

```
method m()
begin
call_soul();
end;
```

14.

```
method m()
begin
f(1, 2 + 3, x);
```

```
end;
```

15.

```
method m()
begin
arr[0];
end;
```

16.

```
method m()
begin
x := arr[i + 1] * 2;
end;
```

17.

```
method m()
begin
begin
x := 1;
end;
end;
```

18.

```
method m()
begin
if 1 < 2 then x := 1;
end;
```

19.

```
method m()
```

```
begin
if x then y := 1; else y := 2;
end;
```

20.

```
method m()
begin
if x then
begin
y := 1;
z := 2;
end;
else
begin
y := 3;
end;
end;
```

21.

```
method m()
begin
if a then
if b then x := 1;
else x := 2;
end;
```

22.

```
method m()
```

```
begin
while x < 10 do x := x + 1;
end;
```

23.

```
method m()
begin
while x < 10 do
begin
x := x + 1;
y := y + 2;
end;
end;
```

24.

```
method m()
begin
repeat x := x + 1; until x > 10;
end;
```

25.

```
method m()
begin
repeat
begin
x := x + 1;
y := y + 2;
end;
```

```
until x > 10;
end;
```

26.

```
method m()
begin
repeat
if x == 0 then y := 1; else y := 2;
until y != 0;
end;
```

27.

```
method m()
begin
repeat x := x + 1; while x < 10;
end;
```

28.

```
method m()
begin
break;
end;
```

29.

```
method m()
begin
while true do
begin
break;
```

```
end;
end;
```

30.

```
method m()
begin
a := true;
b := false;
c := "hello";
d := 'x';
e := 123;
f := 0xFF;
g := 0b10101;
end;
```

31.

```
method m()
var a : array[,] of int;
begin
end;
```

Разработанный модуль протестирован всеми описанными тестами.

Вывод:

В ходе практической работы разработана программа для синтаксического анализа кастомного языка, выбрано ПО для реализации анализа ANTLR3, по формальному описанию составлена грамматика языка. Разработана программа на языке C, которая принимает на вход путь к файлу, содержащему код на кастомном языке, а также опционально путь к выходному файлу/директории. В результате анализа программа создаёт

дерево синтаксического разбора и выводит в консоль, а также сохраняет в .dot файл, который можно визуализировать через Graphviz.